Kernpiler: Compiler Optimization for Quantum Hamiltonian Simulation with Partial Trotterization

Ethan Decker*
University of Pennsylvania

Lucas Goetz
ETH Zurich

Evan McKinney
University of Pittsburgh

Erik Gustafson
(RIACS) at NASA Ames Research Center

Junyu Zhou
University of Pennsylvania

Yuhao Liu
University of Pennsylvania

Alex K. Jones
Syracuse University

Ang Li
Pacific Northwest National Laboratory

Alexander Schuckert
University of Maryland

Samuel Stein
Pacific Northwest National Laboratory

Eleanor Crane
Massachusetts Institute of Technology

Gushu Li
University of Pennsylvania

November 19, 2025

Abstract

Quantum computing promises transformative impacts in simulating Hamiltonian dynamics, essential for studying physical systems inaccessible by classical computing. However, existing compilation techniques for Hamiltonian simulation — in particular, the commonly used Trotter formulas — struggle to provide gate counts feasible on current quantum computers for beyond-classical simulations. We propose partial Trotterization, where sets of non-commuting Hamiltonian terms are directly compiled, allowing for less error per Trotter step and therefore a reduction of Trotter steps overall. Furthermore, a suite of novel optimizations is introduced which complement the new partial Trotterization technique, including reinforcement learning for complex unitary decompositions and high-level Hamiltonian analysis for unitary reduction. We demonstrate with numerical simulations across spin and fermionic Hamiltonians that compared to state-of-the-art methods such as Qiskit's Rustiq and Qiskit's PauliEvolutionGate, our novel compiler presents up to $10\times$ gate and depth count reductions.

1 Introduction

Quantum computing holds immense promise as a paradigm-shifting technology, with one of its most impactful applications lying in *Hamiltonian simulation* [31, 8, 32, 7]—the process of evolving a qubit array according to the physics (Hamiltonian) of a target quantum system. Hamiltonian simulation is widely recognized as a cornerstone of quantum computing's value proposition, as it enables the study of complex physical phenomena that elude classical methods, promising advances in materials science [2], quantum chemistry [5], nuclear- [3] and high-energy physics [11]. However, bringing these benefits to fruition requires efficient compilation strategies to convert the Hamiltonian time evolution to the quantum gate sequences.

Existing efforts in quantum simulation compilation, beyond higher-level compilers such as [43, 34], have employed the domain knowledge and Pauli algebra to optimize the quantum Hamiltonian simulation circuit. In the conventional compilation flow for quantum Hamiltonian simulation (on the left of Figure 1), a Hamiltonian, H, will first be decomposed into a sum of weighted terms, e.g. Pauli strings, $H = \sum_i H_i$ (weights absorbed to H_i's). The Trotter product formula then allows one to approximate the Hamiltonian time evolution e^{iHt} with a long sequence composed of each individual Hamiltonian term, e^{iH_it}, for time evolution. Existing optimization approaches include simultaneous diagonalization of commuting Pauli strings in the decomposition [9, 10, 45], Pauli string reordering optimizations after Trotterization [27, 19, 1], Pauli network synthesis[14, 37], etc., which have yielded noticeable benefits.

The drawback of such conventional compilation flow for quantum Hamiltonian simulation is that each of the Hamiltonian terms must individually be decomposed into its own unitary. Therefore, all these compilation approaches rely on the vanilla error bound in the Trotter formula [20] and focus on reducing the number of gates per Trotter step. Furthermore, to reduce the approximation error one must then increase the number of Trotter steps according to this bound. Notably, for spin and fermionic Hamiltonians, achieving high fidelity with low approximation error typically demands extraordinarily long quantum circuits [4, 7, 21].

The objective of this paper is to show a new path forward for quantum Hamiltonian simulation by incorporating the optimization opportunity from error analysis. We observe that the fundamental bottleneck of product formulas arises from

*Corresponding author. ecd5249@upenn.edu

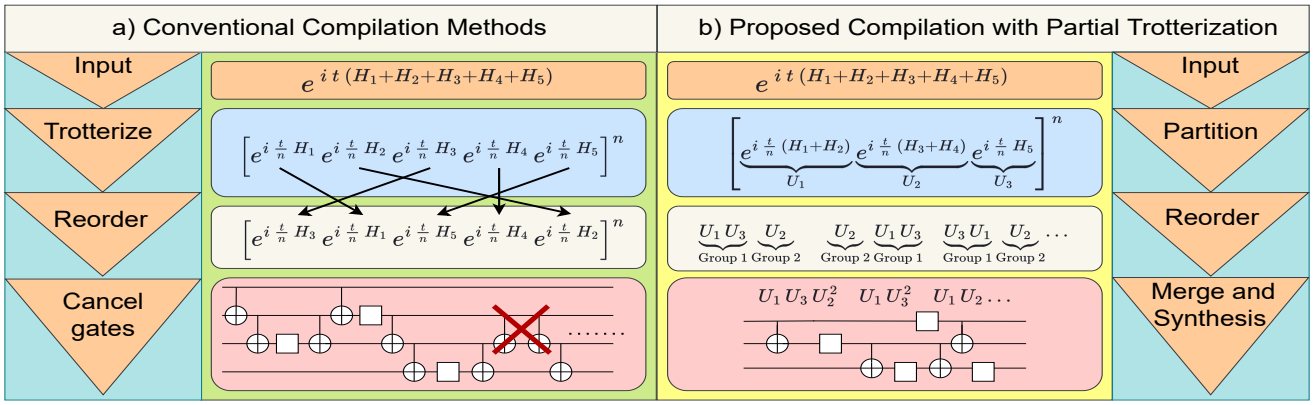

Figure 1: **Conventional compilation flow vs the proposed Kernpiler compiler**. b) Pipeline for reducing gates through error term reduction. First we group into partial Trotter steps which act on a subset of N qubits, in our case N=3. Then we perform an efficient numerical rewrite of the partial Trotter unitaries. Next step, group into commuting subsets of unitaries placing the largest two groups of unitaries on the edges of the Trotter step. Finally, we use a partially symmetric Trotter step to cancel error terms in the expansion by alternating every other Trotter steps order. Commuting unitaries then merge back together naturally allowing for a unitary reduction with no additional error. The compilation finishes at circuit-level.

error scaling, wherein non-commuting Hamiltonian terms are approximated by sequential exponentials. As the error in Trotterization is directly dependent on the non-commutativity of Hamiltonian terms, strategies to mitigate this characteristic in a fine-grained manner can provide a new and scalable way for continued progress in Hamiltonian simulation.

To this end, we propose the new paradigm of *Partial Trotterization* for Hamiltonian compilation, as depicted on the right side of Figure 1. Along with this novel concept, we develop a suite of optimizations, namely Kernpiler, which complement partial Trotterization to command large reductions over modern full Trotterization techniques. **First,** rather than fully decomposing each Hamiltonian term as a separate exponential, we partially Trotter the input Hamiltonian by partitioning non-commuting Hamiltonian terms together into more complex unitaries. We then manipulate and decompose multi-term exponentials instead of exponentials of individual terms. This can significantly improve the error scaling compared with conventional full Trotterization. **Second,** after the partial Trotterization, our Kernpiler groups commuting unitaries together and orders the exponentials of the partially Trotterized Hamiltonian terms to maximize the gate cancellation and term merging. The terms within each group are shuffled at every Trotter step to avoid systematic approximation errors. **Third,** at the final stage, we propose a Monte Carlo Tree Search (MCTS) method to synthesize the exponential of partially Trotterized Hamiltonian terms into a highly optimized basic gate sequence. To maintain the search efficiency, we only search for coupling structures in the MCTS framework, while the single-qubit gates are realized via differentiable methods. This allows us to fully exploit the potential of error reduction from partial Trotterization.

Theoretical analysis shows that Partial Trotterization can effectively lower the Trotter depth (and thus the gate count) needed to reach a desired accuracy, yielding a quadratic reduction in circuit depth as a function of group size for first- and higher-order Trotterization. We also conduct numerical simulation for a range of benchmark Hamiltonians (Heisenberg, Ising, Fermi–Hubbard, etc.) with diverse localities, geometries, and term weights. The results show that Kernpiler outperforms Qiskit's Rustiq [14] and Qiskit's Paulievolutiongate (Paulihedral) [27] with up to a 86% (40% on average) reduction in depth and CNOT gate count along with up to a 85% (11% on average) reduction in single qubit gates (comparing against whichever does better between Rustiq and Paulihedral).

Our major contributions can be summarized as follows:

1. We propose a new decomposition technique, Partial Trotterization, for reducing the error per Trotter step in product formulas.
2. We propose a series of compilation algorithms , Kernpiler, to group the Hamiltonian terms, reorder and merge the grouped Hamiltonian terms, and synthesize the exponential of the grouped terms into basic gates.
3. Experimental results show that Kernpiler outperforms Qiskit's Rustiq [14] and Qiskit's Paulievolutiongate [27] with significant gate count and circuit depth reduction.

2 Background

In this section, we introduce the necessary background to understand the proposed optimization on quantum Hamiltonian simulation. For basic quantum computing concepts (e.g., qubit, gate, linear operator, circuit), we recommend [36] for more details.

2.1 Hamiltonian Simulation, Pauli Strings, and Trotterization

The time evolution of a quantum system with its Hamiltonian H is characterized by the operator e^{iHt} where $t \in \mathbb{R}$ representing the time. In general, directly translating the e^{iHt} into a quantum algorithm is hard and a principled approaches are required.

In this passage we introduce the concept of Pauli string and Hamiltonian decomposition. In an n-qubit system, a Pauli string is defined as a length-n tensor product of the operators $\{X, Y, Z, I\}$, where each operator acts on a specific qubit index. This direct mapping of Pauli strings to qubits naturally arises in many quantum Hamiltonians, making them a convenient basis for both theoretical analyses and practical implementations.

The time evolution of a Pauli string, P is e^{iPt} and it can be synthesized into a quantum circuit using a series of Pauli gates, CNOT gates, and a Z-rotation gate exactly. This process works straightforwardly when dealing with a single Pauli string; however, challenges emerge when the objective is to synthesize an exponential of a sum of Pauli strings, $\exp\big(it\sum_i P_i\big)$. In these cases, closed-form analytical decompositions generally do not exist, which motivates the use of approximation techniques to break down the weighted sum of Pauli Strings into implementable quantum gate sequences.

It is known that all Pauli strings of length n formulate a basis for the linear space of all the Hermitian operators over n-qubits, and Hamiltonians are Hermitian operators. So a Hamiltonian can always be decomposed into a weighted sum of Pauli strings $H = \sum_i w_i P_i$ where $w_i \in \mathbb{R}$. For simplicity, we absorb the weight and the associated Pauli string into one Hamiltonian term and denote $H = \sum_i H_i$ in the rest of this paper. To approximate the exponential of the sum of Hamiltonian terms, one commonly employs *Trotterization*. Formally, it is based on the Lie–Trotter formula [20]:

$$e^{t(H_i+H_j)} \approx \left(e^{\frac{t}{N}H_i}\, e^{\frac{t}{N}H_j}\right)^N, \tag{1}$$

$$\left\| e^{t(H_i+H_j)} - \left(e^{\frac{t}{N}H_i}\, e^{\frac{t}{N}H_j}\right)^N \right\| \leq \frac{t^2}{2N}\,\|[H_i, H_j]\| + \mathcal{O}\Big(\frac{t^3}{N^2}\Big),$$

where N is the number of Trotter steps, and the error depends on the sum of commutators $[H_i, H_j]$ mitigated linearly by the number of Trotter steps. By splitting a large sum into smaller components that can be individually exponentiated, Trotterization provides a systematic method for approximating time-evolution operators. Increasing the number of Trotter steps reduces the approximation error but also increases the overall circuit depth. This method has been implemented in many industry and academia-offered software development kits [22, 16, 25] as a standard approach for quantum Hamiltonian simulation.

2.2 Randomized Compilation

Randomized compilation has recently gained considerable attention in the quantum computing community as a means to mitigate coherent errors in quantum circuits. By converting systematic error into stochastic error, randomized compilation can improve the robustness of quantum algorithm approximations by allowing for better asymptotic scaling on larger time simulations. Early theoretical frameworks for randomized compilation were first presented in [4, 48, 46, 47, 17, 18], illustrating how randomly selected gate layers can effectively reduce correlated noise processes. In product formulas, random compilation can be invoked by shuffling each Trotter step, which would then cause rapidly changing signals and evolutions to average out erroneous terms [6], to give better scaling. This work leverages the idea of randomization to shuffle the orderings of partially Trotterized terms (introduced later) to turn coherent error into stochastic error.

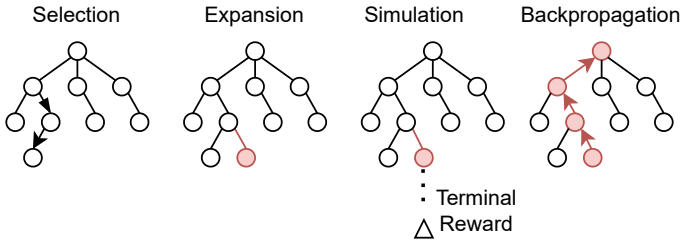

Figure 2: The four stages of the Monte Carlo search tree. 1. Selection of a node for expansion and evaluation. 2) Expansion: choosing a new action and state combination that has not been explored. 3) Simulation: Randomly traversing states and actions to a terminal state and evaluating the outcome. 4) Backpropagation: updating tree metadata on outcomes learned through simulation

2.3 Reinforcement Learning Algorithms and Monte Carlo Tree Search

In this paper, we will also use a reinforcement learning framework to synthesize some unitary operators into basic gates. Here we briefly introduce the framework of the Monte Carlo Tree Search (MCTS) algorithm.

When the structure of a problem is only partially known or highly complex, *reinforcement learning (RL)* offers a powerful framework for decision-making and optimization. It balances the fundamental trade-off between *exploration*—searching for new strategies—and *exploitation*—refining known, successful strategies. Within RL, MCTS is a well-established technique that represents a system in terms of states and actions. To decide which states are valuable and which actions to take to reach valuable states, RL algorithms employ a policy. A policy describes how the algorithm interacts with the environment and is learned over many iterations or attempts.

An MCTS utilizes a tree data structure where actions are represented by edges and states by nodes. The algorithm is fundamentally a Markovian process, where the next action taken is only dependent on the current state. By balancing exploration and exploitation appropriately, our traversal policy should converge to an accurate representation of the value of being in any individual state and therefore allow for a more optimal selection of states and actions over greedy or dynamic programming based approaches.

MCTS proceeds in four key phases (see Figure. 2):

1. **Selection.** From the root of the search tree, MCTS traverses down to leaf nodes following a policy that balances visiting promising states with exploring unvisited ones.

2. **Extension.** At an unvisited leaf, any unexplored actions lead to new states. MCTS selects an action from the leaf and adds the resulting state to the tree.

3. **Simulation.** To quickly estimate the value of this newly added state, MCTS conducts a *Simulation*—a rapid simulation or heuristic-based approximation—until reaching a terminal condition.

4. **Backpropagation.** The outcome of the simulation is then propagated back up the tree to update value estimates and guide future searches.

This iterative process of selection, extension, simulation, and backpropagation allows MCTS to allocate computational effort to promising areas of the solution space while maintaining coverage of unexplored regions.

3 Opportunities and Challenges

Opportunity: Our optimization opportunities come from fine-grained analysis of the error terms in the approximation. The error between the Trotter product formula and exact Hamiltonian time evolution can be shown through the BCH formula [20]. The formula states:

$$\log\left(e^{\Delta t H_i} e^{\Delta t H_j}\right) = \Delta t\, H_i + \Delta t\, H_j + \frac{(\Delta t)^2}{2}[H_i, H_j] + \cdots \quad (2)$$

When approximating $\log\left(e^{\Delta t (H_i + H_j)}\right)$ with $\Delta t H_i + \Delta t H_j$, the dominant error term is $(\Delta t)^2[H_i, H_j] + \cdots$. The higher-order nested commutators are of order $(\Delta t)^3$ and beyond. The primary optimization opportunity identified in this work is to reduce the effect of these commutators. As a small example, consider the following Hamiltonian with 4 terms where none commute with each other:

$$\begin{aligned} H &= H_i + H_j + H_k + H_l, \text{ where} \\ H_i &= X_1 Y_2 Z_3,\ H_j = Y_1 Z_2 X_3 \\ H_k &= Z_1 X_2 Y_3,\ H_l = X_1 Z_2 X_3. \end{aligned}$$

Now, naive Trotterization would give an error of the form:

$$\begin{aligned} \epsilon_{\text{full Trotter}} \propto & [H_i, H_j] + [H_i, H_k] + [H_i, H_l] + [H_j, H_k] \\ & + [H_j, H_l] + [H_k, H_l] \end{aligned} \quad (3)$$

However, if we did not fully Trotterize the Hamiltonian and instead kept $H_i + H_j$ and $H_k + H_l$ in the exponentials (see Figure 1), there would be a smaller bound on the error term:

$$\epsilon_{\text{partial Trotter}} \propto [H_i, H_k] + [H_i, H_l] + [H_j, H_k] + [H_j, H_l]$$

This motivates us to consider grouping terms to contract the additive errors that arise from Trotterization. By strategically partitioning non-commuting operators into commuting partitions, we can potentially reduce the commutator error between terms, leading to lower overall Trotterization error and step counts. However, partitioning the Hamiltonian terms will immediately bring two challenges listed as follows.

Challenge 1: The first question is how we can partition the terms effectively. The objective of partitioning the Hamiltonian terms is to let the partitions be as dense as possible so that the follow-up compilation has more potential to rewrite the circuit with more gate count reduction. Without dense partitions, our rewrites would be very similar to the naive CNOT tree decomposition of the Hamiltonian simulation compilation due to the lack of opportunity for gate cancellations in the rewrite. Existing quantum program partitioning mostly focus on gate-level circuit partitioning for circuit resynthesis [12], [24] which only collects adjacent gates. Other partitionings for specific Hamiltonians have also been explored [33], however, existing partitioning techniques have not been generalized to other Hamiltonians of interest, and often require pre-processing circuits to allow for partitions to be analytically decomposed. Therefore, we believe that there is improvement to be made for Hamiltonian partitioning on the axes of generality and efficiency.

Challenge 2: Suppose we make a partition of Hamiltonian terms H_i, H_j, and H_k. The second challenge is how to efficiently compile and optimize the unitary $e^{it(H_i+H_j+H_k)}$ as there is no established approach for the complicated exponentials. Previous approaches mostly focused on implementing the exponential of individual terms [27], [14], [23]. If we implement the exponential of these terms one by one, we naturally resort to the vanilla Trotterization and lose all the benefits of error reduction from partitioning. Additionally, there exists general unitary decompositions [26], [41], however the gate counts of these methods are very high and can hurt complexity savings from the partitions. Consequently, exploring more efficient approaches for decomposing unitaries is motivated by the hypothesis that using high level Hamiltonian structure and learning algorithms will allow for more efficient circuits.

We now summarize the opportunities and challenges. For conventional full Trotterization, the error at each step is relatively high, leading to a high Trotter step count while implementing the circuit of the exponential of individual Hamiltonian terms is easy. On the other hand, implementing partial Trotterization by partitioning the Hamiltonian terms will reduce the error and thus yield a low Trotter step count while the lack of efficient unitary decomposition methods may negate gates saved through less steps. Overall, our objective is to use partial Trotterization with a new term partitioning method and a new unitary decomposition method for the exponential of many Hamiltonian terms, achieving low Trotterization step count and low gate count in unitary decomposition simultaneously.

4 Kernpiler Framework

In this section, we introduce in detail the Kernpiler framework that can deeply optimize the quantum Hamiltonian simulation by leveraging the optimization opportunities and overcoming the challenges mentioned above.

4.1 Overview

The Kernpiler framework is outlined in Figure. 1b). The input is a quantum Hamiltonian for which the user wishes to obtain e^{iHt} for a set time t.

Firstly, the input is partially Trotterized. For example, instead of fully Trotterizing $e^{i(H_1+H_2+H_3)t}$ to $e^{iH_1t}e^{iH_2t}e^{iH_3t}$, the algorithm may partially Trotterize to $e^{i(H_1+H_2)t}e^{iH_3t}$. To do this, partitions must be formed by sorting Hamiltonian terms

Table 1: Input is an array of Pauli strings. First the algorithm sorts the array on the highest qubit indices acted upon with tiebreakers being the weight of the string. Next the terms are grouped in a greedy fashion such that in each group the terms act on no more than 3 unique qubit indices.

Step	Terms
Input	$[X_3, \quad X_1X_2, \quad Z_3Z_4, \quad Z_1]$
Sort	$[Z_1, \quad X_1X_2, \quad X_3, \quad Z_3Z_4]$
Group	$[Z_1, \quad X_1X_2], [X_3, \quad Z_3Z_4]$
Result	$e^{i\frac{t}{n}(Z_1+X_1X_2)} = U_1, \quad e^{i\frac{t}{n}(X_3+Z_3Z_4)} = U_2$

based on their operator weight (e.g., $X_1X_2X_3$ which acts on three qubits is a weight 3 term), constraining each partition to not act on more than n qubits, where n can be chosen arbitrarily. This results in the dense unitaries labeled U_i in Figure. 1b). Because, in order to do this, the entire circuit needs to be searched, this is the Challenge 1 which we referred to as dense circuit partitioning as discussed in section 3, and which we solve by remaining at a higher level operator representation, referred to as high-level circuit partitioning. Later, these n weight unitaries will be decomposed directly using reinforcement learning methods. Because decomposing arbitrarily high weight unitaries is hard, in the rest of this paper we choose $n = 3$, however we will also comment on choosing larger n later.

Secondly, the partially Trotterized unitaries are grouped such that in each group, the unitaries commute. After constructing groups of commuting unitaries, the order of groups within the Trotter step is determined. For our implementation, two groups containing the most and second most unitaries are placed on the edge of the Trotter step. In every step the side in which the two groups are placed is flipped such that neighboring Trotter steps have at their adjacent edges the identical commuting groups (these will be merged in the following step).

Thirdly, still at the Hamiltonian term level, adjacent identical groups which commute, (i.e., $[U_i, U_j] = 0$) are merged together (i.e., $U_iU_jU_jU_i$ is 'merged' to $U_i^2U_j^2$). After merging groups, there will still be a source of error that comes from the non-commuting terms within a single Trotter step (see Eq. 1). This approximation error would be repeated each time the Trotter step is applied. We refer to this as coherent noise. To counteract this, we randomly shuffle the order of the terms within each successive Trotter step maintaining terms in their respective groups such that this noise becomes stochastic. The k Kernpiler framework then concludes with rewriting the dense unitaries into a target gate set to be executed on a quantum computer.

4.2 Hamiltonian Partitioning Algorithm

The first stage in our compilation pipeline is the partitioning step (shown in Table 1), which allocates Pauli strings into partitions for partial Trotterization. The goal is to maximize the density of terms which do not commute in each partition.

Algorithm 1 Greedy Partitioning Algorithm

Require: List of Hamiltonian terms *Hamiltonian_terms*
Ensure: Partitions of Pauli operators acting on at most 3 qubits
  Sort *Hamiltonian_terms* by their highest qubit index then by term weight
  partitions ← []
  for each *term* in *sorted_terms* **do**
    placed ← *False*
    for each *partition* in *partitions* **do**
      if combined qubits of *term* and *partitions* contain at most 3 qubits **then**
        append *term* to *partition*
        placed ← *True*
        break
    if not *placed* **then**
      append [*term*] as a new partition to *partitions*
  return *partitions*

The input to this Figure is an array of Hamiltonian Pauli terms and the output is partitioned sets of Hamiltonian terms. Currently, each partition of Hamiltonian terms can act non-trivially on 3 qubits maximum, and the unitary made from the partitioned Hamiltonian terms needs to be of size 8×8. Different from circuit-level partitioning strategies, which can only partition a few adjacent gates [12, 24], partitioning the high-level Pauli strings allows us to obtain more dense partitions because many circuit complexities are abstracted away.

Our Hamiltonian term partitioning algorithm is shown in Algorithm 1 and we explain it using the example in Table 1. In this table, the input is the terms of a 4 qubit spin Hamiltonian where each term is weight 1 or weight 2. After receiving the input, the terms are ordered by the largest qubit index acted upon in the term. The terms are then sorted by weight when two terms have an identical max index to define the final ordering.

For example, consider Pauli string $X_1X_2X_3$ and Z_3. The highest qubit index of both terms is shared, and therefore what would decide the final ordering is the weight of the terms (i.e., $Z_3 \leq X_1X_2X_3$). In this sorted order, locally overlapping or anti-commuting terms that should be partitioned together effectively appear near each other, while high-weight or irrelevant terms end up at the tail of the array. In Table 1 we see that Z_1 and X_1X_2 are non-commuting and naturally align close to each other because non-commutation is determined strongly by shared indices. Due to many Hamiltonians being local in nature, sorting by qubit indices tends to put large portions of non-commuting terms very close to each other in the array.

The partitioning phase uses a greedy algorithm which adds terms to the first partition it sees available. If no half constructed partition is available, a new one is created. In our example, Z_1 will invoke a partition creation, X_1X_2 and X_3 will then be added to the same partition. At this point the group is full, so when X_3X_4 is selected next going from left to right, a new group will be created to avoid having more than 3 unique indices in one group. The resulting partitions tend

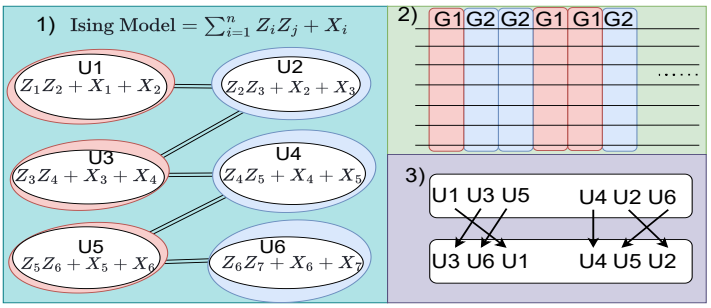

Figure 3: 1) **Create Groups:** A conflict graph is constructed showing commutation relations of Hamiltonian terms. A vertex indicates a unitary of the Trotter step. An edge indicates that two unitaries do not commute. Independent sets are created about the graph which are used to group unitaries with other pairwise commuting unitaries. 2) **Order Full Groups:** The groups created are ordered in the Trotter step for cancellation with other groups. The two largest groups are placed on edges of the Trotter step. At the neighboring Trotter steps, the groups placed at the edges swap places such that identical groups are neighboring each other. Unitaries are then merged via commutation equivalences. 3) **Shuffling Group Term Order:** The order of terms within each group is shuffled to invoke stochastic noise over coherent noise.

to be dense enough to allow meaningful circuit optimizations while also maintaining simplicity.

4.3 Trotter Step Reordering and Randomization

In the second stage of our optimization pipeline, we reorder and randomize our partially Trotterized unitaries (see Figure 3). Here, we use a simple input case, the 1D Ising model. The input consists of a set of partially Trotterized unitaries of the form $e^{i(\sum H_i)t}$, which together form a single Trotter step. The Pauli strings contained in each unitary are also listed. In Step 1, we construct a conflict graph that represents the commutation relationships between the Trotter step unitaries. These unitaries are generated as outputs from the previous algorithm described in section 4.1. Independent sets, corresponding to mutually commuting unitaries, are then extracted from this graph to form commuting groups. The two independent groups are denoted as G1, and G2 respectively, in Figure 3. Extracting independent sets is done in a greedy fashion according to Figure. 3. After identifying independent sets, Step 2 shows the ordering of groups within 1 Trotter step. Groups are ordered such that with neighboring Trotter steps, identical groups are neighboring each other and can be trivially merged into fewer unitaries; this is beneficial for the final output.. For example, imagine e^{iH_it}. Due to all of the terms mutually commuting, the identical unitaries can be reordered such that $e^{iH_it}e^{iH_it} \rightarrow e^{i2H_it}$ which reduces the unitary count from the perspective of mapping unitaries to gates. Step 3 we mitigate coherent noise by shuffling the order of unitaries in each group. Notice that the ordering is not shuffled between groups, and that all unitaries stay within their assigned group from Step 1. This approach effectively reduces the overall circuit depth and gate complexity, optimizing the quantum circuit compilation without incurring additional approximation errors.

Here we describe how to obtain the groups found in Step 2 of Figure 3. The greedy independent set algorithm, described in Algorithm 2 starts with the conflict graph as input. Starting with a vertex, for example the vertex with the lowest index, add all vertices not sharing an edge with the target vertex to our group. Second, we need to remove all vertices in our newly formed group from the conflict graph so that these vertices are not repeated in newer groups. The process is then iterated again to get the second largest maximally independent set of the graph. The conclusion of this algorithm outputs two sets which are to be merged with their identities on the boundaries of Trotter steps, as seen in Figure 3, Step 2.

Algorithm 2 Trotter Step Reordering and Randomization

function BUILDCONFLICTGRAPH(H)
  Initialize graph $G = (V, E)$ where each node $v_i \in V$ corresponds to a term in H
  for each pair of terms (t_i, t_j) in H **do**
    if $[t_i, t_j] \neq 0$ **(they do not commute) then**
      Add edge (v_i, v_j) to G
  return G

function GREEDYCOMMUTINGGROUPS(G)
  $groups \leftarrow []$
  while G is not empty **do**
    $I \leftarrow$ GreedyMaxIndependentSet(G)
    append I to $groups$
    Remove nodes in I (and their edges) from G
  return $groups$

function REORDERTROTTERSTEPS($\{H_1, \ldots, H_n\}$)
  for each Trotter step H_k **do**
    $G_k \leftarrow$ BUILDCONFLICTGRAPH(H_k)
    $groups_k \leftarrow$ GREEDYCOMMUTINGGROUPS(G_k)
    randomize the ordering **within** each group in $groups_k$
    concatenate commuting groups contiguously
  reorder consecutive Trotter steps
  merge commuting operators across adjacent steps where possible:
  if $[A, B] = 0$ for A in step k, B in step $k{+}1$ **then**
  return {modified Trotter steps}

4.4 Unitary Decomposition for Grouped Hamiltonian Terms

After we group the Hamiltonian terms and order them, the final step is to decompose these grouped terms into basic gates. As discussed in section 3, the key to successfully leveraging the benefit from partitioned Hamiltonian terms is being able to efficiently decompose the exponential of the partitions into basic gates. An MCTS is an algorithm designed to handle sequential decision problems where there is little information about the environment, which is exactly the problem of circuit synthesis for general combinations of Hamiltonian terms. With a good balance of exploring new solutions and exploiting known working solutions, performance can be

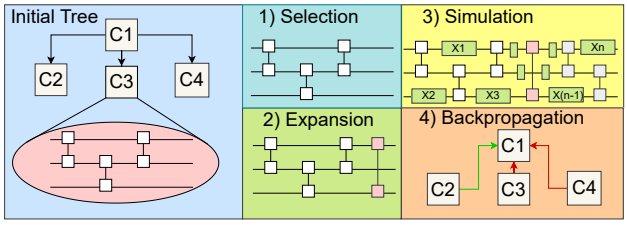

Figure 4: **Unitary decomposition method** 1) **Selection:** Select a node in the search tree which represents a partially synthesized circuit which has unexplored child actions. 2) **Expansion:** Select a CNOT gate among choices from the gateset to append to the circuit. 3) **Simulation:** Starting from the newly expanded state, append CNOTs until we reach a terminal circuit length. After, interleave a fixed number of single qubit gates at random in between the CNOT gates. Optimize parameters with the Gauss-Newton method. 4) **Backpropagation:** Update values of nodes in the tree based on the result of the simulation stage to identify if the newly explored state was valuable.

better than greedy heuristics and have more flexibility than dynamic programming-based approaches.

An example of how MCTS elements fit into our framework is shown in Figure 4. Referring to the initial tree in the example, each tree node state is a circuit of strictly CNOTs. Actions the algorithm can take are defined as CNOT gates which can be appended to a partially synthesized circuit expressed by a node state. The MCTS algorithm starts with the **selection** process. The goal of selection is to find a promising node of the tree data-structure for which actions taken from that node state have not been explored yet. During our selection process, we traverse the tree using a policy until we reach a node with unexplored actions. The circuit shown in blue is the partially synthesized circuit for which the node selected represents. In the **expansion** step, an unexplored action is explored which leads to a new node being appended to the tree as a child to our selected node. The difference now is a CNOT gate has been appended to our selected node state, creating a new state that has no known value yet. In the **simulation** step random CNOT gates are then appended to the circuit. Following the appending of random CNOT gates up to a fixed circuit length, single qubit gates are then interleaved between all CNOTs. The result is the circuit diagram shown in the simulation step of Figure. 4. After generating a fully synthesized circuit, parameters of single qubit gates are solved for such that the values minimize the error between the synthesized circuit and the target unitary. The value of the state is then determined by the amount of CNOT gates and the error of the approximation. At the end of our algorithm, the fully synthesized circuit with the largest value is returned. **Backpropagation** is the final stage of the algorithm where the value of each state is updated based on the results of the simulation stage. In the example, three partial circuits were evaluated and the values of the results are passed from the leaf nodes to the root node. This allows the algorithm to learn and make better decisions on future iterations.

To select a node, a key tradeoff in the field of reinforcement learning is the balance of exploiting known solutions and exploration of new solutions that may lead to better results. The selection of a node to explore is determined by a policy. A policy in general context is how the algorithm decides which actions to take. For our policy, the input would be the value of nodes to traverse to and the number of times the nodes have been explored. The output is a decision of which action to take leading to the state deemed most promising by the policy. In Monte Carlo search tree, a common policy for this purpose is the canonical UCT policy [44].

Why focusing on CNOT skeletons. When designing the MCTS algorithm, we must consider the concept of coverage. Coverage is a quantity that says how much of the search space can be covered by the RL algorithm. Quantitatively, coverage can be defined as follows:

$$\text{Coverage} \approx \frac{N_{\text{visited}}}{b^{D_{\max}+1}} \tag{4}$$

Here, N is the number of states visited, b is the number of choices available at each state (the branching factor), and D is the maximum possible length of a sequence (a circuit in our case). In order to make the environment more tractable, our depth and branching factor should be as small as possible which will give more coverage of the search space to find an optimal solution. In the straight forward framework of circuit synthesis, our branching factor (due to choice of angles) and the depth of a circuit can be impractical for light weight RL agents due to continuous action spaces and very long depths.

A key insight to our algorithm design is that we only consider CNOT gates when defining states of the partially synthesized circuit. The motivation was out of necessity to condense the search space of synthesizing a circuit where the search space is defined by all permutations of a universal target gate set. The intuition is that the entanglement structure is the most difficult characteristic to solve in circuit synthesis and that single qubit gates that are continuously parameterized can lead to a smooth landscape for optimization via differentiable methods. For our approach, once an entanglement structure is determined, the circuit is overparameterized with many single qubit gates injected at all circuit layers. Overparameterization is important because it leads to a smoother cost landscape compared to a function with fewer parameters. Using the Gauss-Newton method, we minimize the L2 norm, our cost function, of the difference matrix between the target and approximation circuit. After getting an optimized solution, all strings of single qubit gates can be rewritten as one single qubit gate making the circuit optimal for quantum hardware. For our implementation, the Qiskit transpiler at level 3 optimization is used to convert our overparameterized circuit into an optimal circuit expressed in the (u3,cx) gateset.

Value of our simulated solution is calculated as a function of accuracy and gatecount (Eq. 2). The function is non-continuous and depends on the accuracy of the circuit being above or below a threshold error, which we have set to 10^{-8}. If the error of the approximation after simulation is below this threshold, value is determined strictly by the negative of CNOT gate count. However, if the error of the approximation is above the threshold, value is determined strictly as negative error. For example, if the circuit in Simulation of Figure. 4 had an error of below 10^{-8}, then the value would be -6. However, if the error was above the threshold, the value

Table 2: Benchmark information with grid sites and final qubit counts (for Fermi–Hubbard, qubit# = 2 × grid sites)

Benchmark	Topology	Size	Qubits
Fermi-Hubbard (FH)	Triangular Grid	2×2	8-128
	Square Grid	2×2	8-128
	1D Grid	5×1	10-144
Heisenberg (HB)	Triangular Grid	5×2	10-144
	Rectangular Grid	5×2	10-144
	1D Grid	10×1	10-144
Ising (IS)	Triangular Grid	5×2	10-144
	Rectangular Grid	5×2	10-144
	1D Grid	10×1	10-144
LiH Molecule (LiH)	Molecular	N/A	10
HF Molecule (HF)	Molecular	N/A	10
PD-1 Protein (PD1)	Molecular	N/A	28-222

would be $-\epsilon$.

$$\mathcal{E}(x) = \text{argmin}_\theta || \prod_{i=1}^{n} x_i(\theta_i) - U||_2 \tag{5}$$

$$R(x) = \begin{cases} -\#\text{cnot}, & \mathcal{E}(x) < \epsilon \\ -\mathcal{E}(x), & \text{otherwise} \end{cases} \tag{6}$$

The intuition is that there will be important information, referred to as a signal, given even in the event of failed simulations to tell the algorithm where more and less accurate solutions are occurring. After finding solutions over a threshold, accuracy offers diminishing returns and gatecount becomes a larger priority. Backpropagation is then simply performed by updating all Q_i from the UCT policy for each node that has been traversed in the selection phase. If a winner is found, in practice many are found at once, then the best circuit is returned immediately.

5 Evaluation

Experimental Configuration: To evaluate our work, we measured performance using metrics with and without error scaling accounted for. For error quantification, we compare the approximate unitary with the theoretical perfect unitary using the L2 norm of simulation Hamiltonians that involve between 8 and 10 qubits. The L2 norm has been commonly used to quantify error of approximations in quantum algorithms [13, 7] and therefore that is our metric of accuracy here. Our target is to compile results to greater than 99.5% accuracy. We also perform Trotterization comparisons to evaluate error reductions in both near-term and long-term applications. For the second order Trotterization, we use a time simulation with t = 1 in dimensionless units, and our experience shows that for significantly longer simulations, the second order method performs markedly better for most general tasks compared to the first order Trotterization. In contrast, for the first order Trotterization, we consider short time simulations by scaling all Hamiltonian coefficients by t=0.1 which is appropriate for near term applications to observe short time dynamics of quantum systems. It is important to note that Qiskit's PauliEvolutionGate currently defaults to first order Trotterization; therefore, we recommend viewing the corresponding chart for a more accurate state-of-the-art comparison and review the second order for future more general use of quantum computers for quantum simulation. Scalability is assessed by measuring runtime and gate count using 28-220 qubit Hamiltonians. For the larger Hamiltonians, the L2 norm cannot be measured however we expect the same reduction in error at larger sizes because the weights of terms do not increase with system size for most Hamiltonians. We discuss the scalability in Section 6

Software and Hardware Setup: Our implementation is carried out using PyTorch version 2.5.1+ CUDA 12.1, and we compare our results against Qiskit's stable version 1.3.2, which features state-of-the-art Hamiltonian compilation methods inspired by the works of Rustiq [14] and Paulihedral [27]. The hardware setup includes an A100 GPU with 80GB of RAM for implementing the Monte Carlo search tree, alongside an AMD EPYC 9654P 96-Core Processor for the overall implementation. Furthermore, for evaluation of our Monte Carlo Unitary Synthesis, we compare against some existing the unitary synthesis toolkits in Qiskit stable version 1.3.2 and BQSkit [35] version 1.2.0.

For circuit generation, we create Qiskit circuits for all algorithms, including our proposed method, the paulievolutiongate, and the paulievolutiongateRustiq. In the case of first-order Trotterization, we employ the Lie-Trotter formula, modifying only the number of steps from the default configuration. For second-order Trotterization, we use the Trotter-Suzuki formula with the same adjustment in the steps argument. After circuit generation, we optimize the circuits at level 3 optimization in Qiskit's transpiler using the u3 and CNOT basis with all-to-all connectivity. The optimized circuit is then converted into a numerical format to calculate the L2 norm of the difference matrix, and by squaring this norm, we estimate the order of magnitude on state fidelity.

Benchmarks: To ensure a comprehensive evaluation, we select a wide range of popular Hamiltonians that vary in topology, geometry, terms, and correlation structures (see Table 2). For nearest neighbor models, we include the Ising and Heisenberg models, which demonstrate varying site densities (the number of Hamiltonian terms per site). Additionally, we consider non-local models, such as the Fermi-Hubbard model and molecular Hamiltonians, where variations in correlation and dimension help expose the strengths and weaknesses of the different compiler methods. All fermionic models have been mapped to qubits using the Bravyi-Kitaev mapping [40]

5.1 Comparison with Baseline without Considering Error Reduction

Firstly, we will discuss the absolute data comparisons in terms of gate count and circuit depth when using various compilation techniques to quantify compilation efficiency per Trotter step rather than the accuracy of the compilation. That is, the compilation error reduction is not included and will be evaluated later in the next section. Therefore, different from other experiments, our Hamiltonian sizes are of range 28-220 qubits, and the L2 norm was not considered. For competitive benchmarks, we compare against two state-of-the-art unitary synthesis techniques: BQSkit and Qiskit's unitary synthesis, and two state-of-the-art fully integrated Hamiltonian com-

Kernpiler | QiskitPauliEvolutionGate | RustiqPauliEvolutionGate | Qiskit Synthesis | BQSkit Synthesis

Test Case	Size	Depth	CX	U3	Time	Depth	CX	U3	Time	Depth	CX	U3	Time	Depth	CX	U3	Time	Depth	CX	U3	Time
FH 1D	64	613	2113	1993	313	1081	2016	1536	0	6450	9404	6997	3	989	3148	3471	7	828	2872	3144	302
FH 1D	144	1055	4998	4632	625	4017	4647	3537	2	19807	31961	20013	14	1302	7570	8337	16	1190	7037	7614	438
FH 2D	50	2664	5172	3267	215	5368	4900	2757	2	9547	13421	12322	5	3169	5927	4281	9	3025	5753	4117	281
FH 2D	128	4378	14101	8827	644	7528	13236	6921	6	58591	91874	85560	50	4467	16154	11808	29	4441	15621	10965	462
FH Tri	50	4911	8343	4952	247	8868	7780	4083	4	15230	21257	19510	5	5129	9231	6134	15	4904	9054	5881	283
FH Tri	128	7528	23889	13287	682	14857	22206	10392	10	96534	146k	134k	64	7927	25897	16410	54	8152	25578	15700	990
HB 1D	64	72	566	1172	117	403	567	1198	0	744	1367	785	0	125	1120	1917	5	117	805	1625	630
HB 1D	144	71	1284	2649	262	883	1287	2718	0	1404	2967	1625	1	125	2560	4377	11	119	1845	3696	1230
HB 2D	64	256	1224	2459	165	216	1008	2024	1	8633	11622	10936	3	517	2447	4124	7	335	1770	3473	813
HB 2D	144	240	2914	5870	385	312	2376	4764	2	47151	64672	62567	27	555	5813	9791	18	353	4241	8392	1718
HB Tri	64	221	1275	2573	178	322	1449	2771	1	17360	23052	21947	6	475	2431	4091	8	401	1850	3642	765
HB Tri	144	273	2935	5895	391	462	3465	6599	3	105k	142k	141k	73	519	5816	9774	19	401	4072	8103	1502
HF	10	1200	1073	847	83	1455	1200	700	0	562	688	446	0	1568	1344	1201	2	1631	1383	1289	216
IS 1D	64	43	324	708	599	204	378	381	0	262	566	379	0	113	996	1789	4	113	976	1760	289
IS 1D	144	43	731	1595	1383	444	858	861	0	604	1278	861	0	113	2276	4089	10	113	2270	4081	448
IS 2D	64	128	678	1260	1962	111	672	528	0	806	1842	507	0	328	1817	2596	5	298	1808	2582	286
IS 2D	144	123	1479	2924	1018	159	1584	1224	0	1130	3827	1175	1	358	4287	6064	12	335	4278	6050	439
IS Tri	64	191	994	1971	507	204	966	675	0	890	1956	653	0	334	1890	2746	5	428	1693	2503	455
IS Tri	144	198	2111	4217	828	294	2310	1587	0	1085	4401	1530	2	388	4149	6031	14	363	4002	5835	805
LiH	8	1205	1084	914	59	1455	1200	700	0	561	688	445	0	1480	1388	1226	2	1617	1324	1190	228
PD1	28	574	1082	740	85	500	945	477	0	539	717	413	0	766	1310	1150	2	783	1268	1079	253
PD1-ext	74	4857	10443	6458	506	6878	9976	4545	4	17297	24339	21241	10	4996	12408	8816	24	5130	11891	7965	514
PD1-super	222	26804	83423	45507	2141	54603	78499	31571	45	472k	638k	593k	1332	27603	92733	56335	344	26775	90005	52741	2211

Figure 5: Absolute data comparisons with no error reduction considered. Hamiltonians are compiled to a fixed number of Trotter steps.

pilation schemes: Rustiq and Paulihedral. For the unitary synthesis, we replaced the Monte Carlo Search Tree algorithm from Section 4 with BQSkit and Qiskit to compare ablated performance to our full Kernpiler approach. For metrics compared, each Hamiltonian was compiled to 3 Trotter steps, for which we recorded the quantity of U3 and CNOTs within the end-to-end circuit. Other metrics besides gatecount measured were the system size, compilation time (seconds), and total depth of the circuit. This data is summarized in Figure 5.

We now summarize our interpretation of Figure 5. Our MCTS-based approach achieves the smallest gate counts among the other unitary decomposition techniques without having the longest runtimes, akin to BQSkit, for which we cached the compiler to save on time. More specifically, compared to the ablated MCTS pass, our results achieve a peak(average) reduction of U3 gate counts by 60% (33%), CNOTs by 67% (34%), and depth by 60%(32%) compared to the next best performer for each benchmark. The reason BQSkit and our MCTS-based method have longer compilation times compared to other existing techniques is due to the fact that BQSkit and our methods are search-based, where the algorithms will try different decompositions in a large search space to try and find an optimal decomposition. Other approaches are based on constructive heuristic algorithms. For a breakdown of runtime per pass, see Figure 7.

Our approach outperforms BQSkit in runtime by leveraging implicit bias in the search space. By having high-level information given to the MCTS, our compiler can perform a more reasonable search of available options and can also set obtainable optimization goals (such as searching for a 10-gate solution vs a 5-gate solution). Additionally, BQSkit and Qiskit both have the issue that they are performing unitary synthesis at the lowest level of representation and cannot assume a high-level structure of their input, allowing for goal setting, which reduces the search space.

An exciting result is that our technique has advantages even when excluding the Trotter steps saved. Furthermore, our technique still achieves a peak(average) CNOT reduction of 14% (0%) and a depth reduction of 60%(14%) compared to the next-best-performing technique for each benchmark. The Kernpiler design also manages to close absolute gate performance gaps as we increase the size and complexity of our Hamiltonian. For example, our most complex benchmark (PD-1 protein fragments) has a large performance gap for small sizes of 28 qubits and progressively closes the gap in gate count and achieves the best depth metric in the largest fragment of 222 qubits. Further, we point out a weakness in the U3 count, without considering Trotter step reduction, U3 gate count increases by 50% on average compared to the next best performer, but is handled by saved Trotter steps which we show later.

A reason for our results in depth is due to the design. By not having optimizations primarily focused on reordering the Trotter step, which invokes a tradeoff between depth and gate count, our stack can optimize along the axis of depth and gate count concurrently. Therefore, while partial Trotterization and MCTS optimize gate count, reordering can be used to optimize depth, which avoids the original tradeoff.

We now discuss a limitation of our design regarding the lack of optimization towards single qubit gates. There are two systemic reasons for this. Firstly, our MCTS is optimizing over CNOT count rather than total circuit complexity, as illustrated by the cost function of Equation 6. Secondly, we intentionally over parameterize the circuits with single qubit gates to gain convergence. This over parameterization is then optimized via existing compilation techniques. The combination of injecting complexity into the circuit and not designing the search space to express single qubit gates is contributing to the lack of performance on this metrics. Such limitation can be mitigated by more complex search heuristics which include U3 gates as an optimization target.

5.2 Comparison with Error Measurements

We now discuss the comparison of our compilation software against competitive benchmarks focused on Trotterization optimization when considering error savings. Figure 6 presents the results for first-order Trotterization (Lie–Trotter) and second-order Trotterization (Trotter–Suzuki). The graphs are normalized to display percentage reductions from the maximum gate count(depth) observed. Overall, the data reveals a higher reduction for the first-order Trotterization compared to the second order, which still achieves about a three-fold reduction in the best-case scenarios for gate count and up to a 10x reduction in depth.

Two primary factors account for the difference between first and second-order improvements. First, the constant factor in our commutation relation is reduced by a square root for the first-order Trotterization. Specifically, while the first-order Trotterization scales as $\Delta t^2/N$, the second-order scales as $\Delta t^3/N^2$ where N is the number of Trotter steps. Consequently, a constant reduction factor in the numerator, which is what partial Trotterization enables, will be diminished by an N^2 step scaling in the second order case, whereas the first order requires a linear number of steps, as compared to a square root number of steps, to reach the same level of optimization. Second, for bipartite Hamiltonians—those whose conflict graphs from Section 4.2 are bipartite—the two commutator groups span a large portion of the Trotter step. Because the order of commuting groups is reversed in these cases, an almost Δt^3 scaling can be observed. This behavior, evident across a wide range of benchmarks, is attributed partly to system size and partly to the high degree of commutativity. These effects also explain why, in the second-order data, the reduction does not reach the square root improvement observed in the first-order Trotterization, as the competition scales more appropriately with our method. Additional observations include the performance differences among the various compilers.

Qiskit's PauliEvolutionGate tends to perform best on very regular, low connectivity, low weight Hamiltonians, while Rustiq performs , by design, optimally on molecular/electronic structures with non-trivial connectivities and terms. The largest gap in performance is observed in cases with non-trivial yet regularized connectivities, such as the triangular lattice and electronic Hamiltonians with long-range correlations over a symmetric lattice. Additionally, our compiler tends to perform very well on Hamiltonians that are denser in terms per site (i.e the Heisenberg models vs the Ising models). This outcome can be attributed to the nature of our optimizations; relatively local connectivity—even in the presence of non-trivial topologies—allows our grouping algorithm to identify large commuting sets, and our rewrite procedures, being independent of other Hamiltonian terms, are less affected by unpredictable correlations. Notably, Rustiq appears to underperform on most non-electronic Hamiltonians. In contrast, PauliEvolutionGate serves well as a general spin Hamiltonian compiler, excelling on symmetric local connectivity but struggling with irregular patterns, as evidenced by its performance on electronic structure Hamiltonians and the atypical topologies found in local/power law Hamiltonians.

For the Ising models, an interesting discrepancy is observed: while the CNOT gate count is extremely low, the U3 count is significantly higher. This is because our rewrite system does not employ a CNOT tree or chain for decomposition. As a result, more U3 unitaries appear in odd or sandwiched locations, whereas a CNOT tree decomposition would eliminate the need for basis changes and require only a single Z gate, thereby intrinsically reducing the U3 count.

6 Error Reduction Theoretical and Experimental Data

Here we offer a theoretical explanation for the error reductions observed, alongside an understanding of how this concept scales to larger rewrite radii and lattice size. Theoretical error reduction fundamentally arises through commutator cancellations. To illustrate this, we start from the standard derivation of Trotterization, where the error terms can be expressed as a sum of commutator norms:

$$\text{Error} = \sum_{i<j} \frac{|[H_i, H_j]|}{2} \Delta t^2 + \mathcal{O}(\Delta t^3). \tag{7}$$

By partitioning Hamiltonian terms, we instead consider commutators between entire groups rather than individual terms, leading to:

$$\text{Error partitioned} = \sum_{A<B} \frac{|[H_A, H_B]|}{2} \Delta t^2 + \mathcal{O}(\Delta t^3) \tag{8}$$

where each group H_A is composed of individual Hamiltonian terms maximized for non-commutativity. Importantly, the commutator between partitions $[H_A, H_B]$ is simply the aggregation of all individual commutators $[H_i, H_j]$ where $H_i \in H_A$ and $H_j \in H_B$. Thus, the partitioned error (Eq. 8) explicitly represents the original error minus the intra-group commutator contributions that vanish due to partially Trotterized unitaries. This leads to a final reduced error of Trotterization to:

$$\text{Error reduced} = \text{Error} - \text{Error grouped}, \tag{9}$$

quantifying the precise error savings achieved through term partitioning and highlighting the scalability of this methodology. As the partition size increases, the number of intra-partition commutators grows combinatorially, scaling roughly as n_A^2 for a partition of size n_A. Consequently,

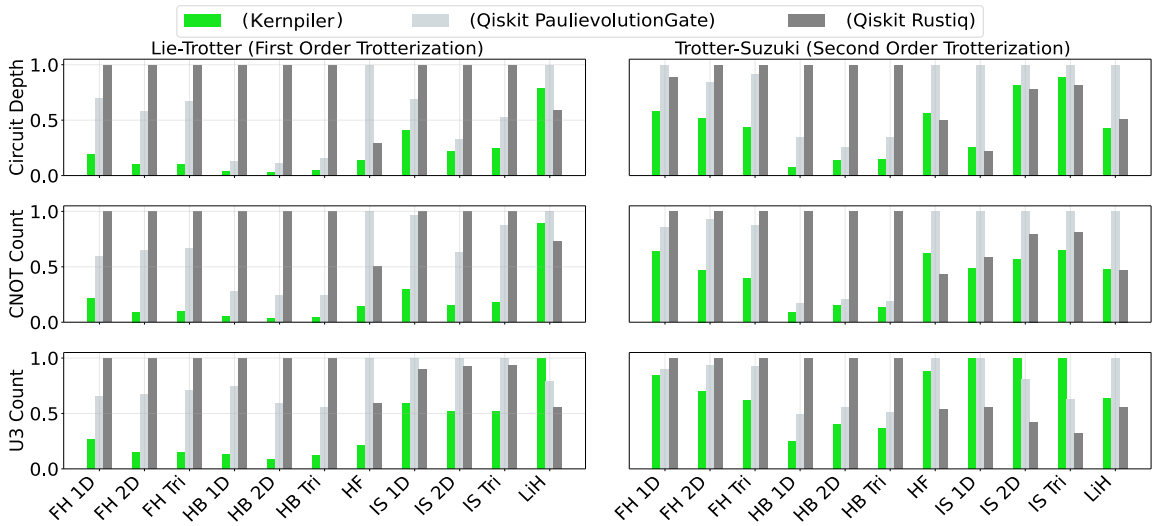

Figure 6: Depth, CNOT and U3 count comparison when compiling to less than 1% approximation error on a range of time evolution unitaries.

Test Case	Size	Partition	Reorder	Rewrite	Total
FH 1D	64	0.003	0.013	2644.173	2704.220
FH 2D	50	0.002	57.530	3116.759	3236.742
FH Tri	50	0.003	55.665	3139.404	3206.768
HB 1D	64	0.002	0.002	116.388	117.043
HB 2D	64	0.006	0.005	160.582	161.790
HB Tri	64	0.009	0.006	163.014	164.746
HF	10	0.000	0.013	637.678	696.476
IS 1D	64	0.001	0.002	599.855	600.327
IS 2D	64	0.003	0.004	509.389	510.116
IS Tri	64	0.005	0.005	520.807	521.738
LiH	10	0.000	0.013	58.921	60.359
PD1	28	0.001	0.006	78.796	80.077
PD1-ext	74	0.016	0.266	540.464	555.245
PD1-super	222	0.496	159.378	5396.862	5885.737

Figure 7: Runtime (in seconds) for all passes of Kernpiler when compiling large benchmarks

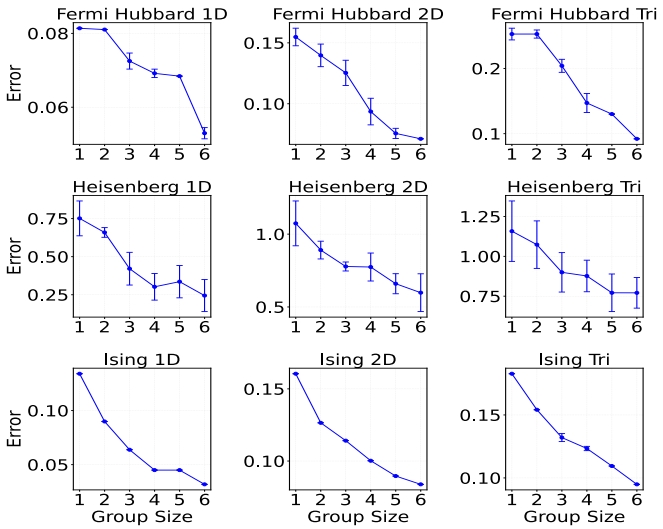

Figure 8: Increasing the number of qubits per unitary to decompose directly reduces the error.

error reduction becomes significantly more pronounced as larger partitions are formed, since more commutator terms vanish. Thus, increasing the rewrite radius directly enhances error reduction, emphasizing the scalability and efficiency of this partial Trotterization approach in practical quantum simulations.

We investigated this empirically with first order Trotterization of Hamiltonians decomposed using 10 Trotter steps with no special optimizations. The only change over decompositions is the amount of partial Trotterization performed. In Figure 8, we show scaling of the compiler error versus group decomposition size (number of qubits) across 3 different models with 3 different geometries. We performed 5 runs per data point. The remarkable find is that the approximation error decreases drastically as a function of group size; this highlights a remarkable benefit of the partial Trotterization schema.

The Ising models possess the monotonic trends which are likely an artifact of the simple distribution of Hamiltonian terms that allows for easily converging on the best partitions. For the other models we see more significant effects from noise. This originates from the partitioning of Hamiltonian terms. As the entanglement structure becomes increasingly non-trivial, the partitioning algorithm encounters greater difficulty converging to the optimal partitions causing more noise in the commutation error observed.

Scalability Discussion Now we discuss the scalability of our technique to larger quantum lattices and Pauli weight terms. Modeling the exact error is intractable as we scale qubit size, however Trotter error is directly proportional to the amount of non-commuting pairs of terms which define the Hamiltonian (see Equation 7).

Figure 9 shows the amount of non-commutivity as we increase qubit size. In this experiment, we graph the ratio of non-commuting pairs between a partitioned and unpartitioned Hamiltonian. More specifically, we graph $\frac{\#\text{non commuting pairs partitioned}}{\#\text{ non commuting pairs}}$. Partition sizes used were $n = 3$

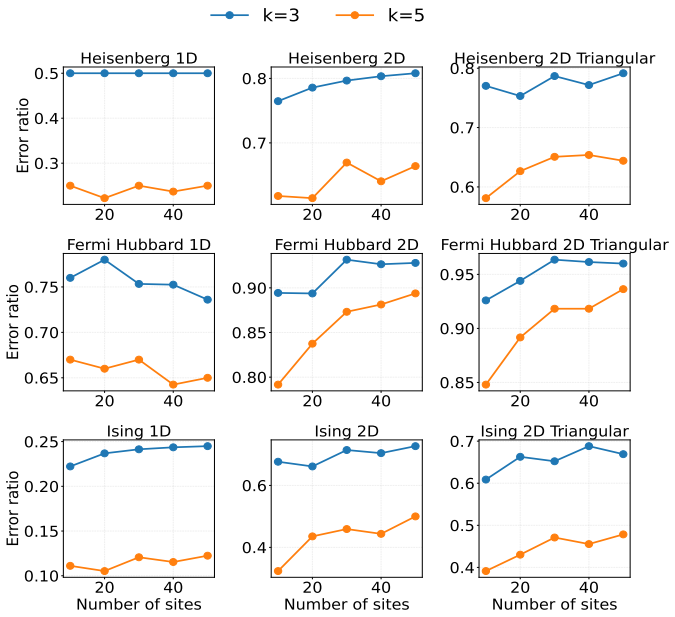

Figure 9: Ratio quantifying percentage reduction of non-commuting pairs of Hamiltonian terms between partitioned and unpartitioned Hamiltonians over increasing quantum lattice sizes.

and $n = 5$. This ratio is measured over system sizes from 10 to 50, extending out from our benchmarks measured in Figure 6. At each qubit array size, we measure the ratio of non-commuting pairs, seen on the Y axis. The expected behavior is that for k-local Hamiltonians, the non-commutation ratio should not significantly increase. The reason for this expectation is that while more terms are being added, their weight is not increasing. As a consequence, these terms can also be fit into new partitions which reduces the error relative to an unpartitioned Hamiltonian. This implies our technique—and improvements made to the partition size—would continue to have a constant rate decrease in total Trotter error that is independent on array size. This is the exact behavior seen in the data of Figure 9. For the electronic structure Hamiltonians, the fermion to qubit mapping used was the Bravyi-Kitaev mapping [40]. The weight of Pauli terms increases logarithmically, so in this experiment, the expected behavior is a logarithmic curve. This is because logarithmically, terms are being added which cannot fit into our partition size (n=3,5). This however can be mitigated as techniques exist to have constant weight pauli terms [15] We notice that there is noise in some of the ratios and we believe this is an artifact of the partitioning heuristics used. Overall, this proxy measurement gives evidence towards the scalability of constant-size partitions for reducing Trotter error as the ratio of non-commutation appears to have no dependence on quantum lattice size.

7 Related Works

Trotterization error has been extensively studied, resulting in various strategies aimed at mitigating and managing these errors. Gui et al. [19] demonstrated that grouping neighboring terms in the Trotter step ordering can reduce errors by effectively clustering commuting operations. Additionally, a recent survey shows early work for rewriting certain partitions of the Hamiltonian to save on error per Trotter step [33]. However, these partitions are not general to all Hamiltonians of interest, and are also restricted to unitaries with special properties, making adaptability to input very difficult.

Theoretical advancements, including higher-order Trotter decompositions [4], systematically eliminate specific-order errors through symmetric expansions. Our method can provide better performance due to the partial Trotter decomposition. By rewriting the non-commuting terms with minimal error, the error bound is reduced, which complements the optimizations and techniques described above in practice.

Compiler optimizations for quantum Hamiltonian simulation, outside of error reduction, have also been extensively studied. Simultaneous diagonalization of commuting Pauli strings [9, 10, 45] is one early type of approach. They are later outperformed by reordering-based gate cancellation [27, 19, 1] and Pauli network synthesis [14, 37, 39].

Similar work casted reordering and synthesis of the Trotter step as a travelling salesman graph problem, [39] which was able to reduce the depth of Trotter steps substantially. Unlike our goal of grouping by commutation for merger across Trotter steps, the authors of this work framed Trotter reordering for optimization within a singular step.

The recent work QuCLEAR [28] investigated extraction and absorption for Clifford gates in quantum Hamiltonian simulation, but it requires updating the observable. This work does not change other parts of the circuit, and the compiled Hamiltonian time evolution operator can be freely reused. Moreover, all of them rely on the vanilla error bound of Trotterization and do not consider the fine-grained error scaling. Finally, other works focused on fermion to qubit mappings for cancellation of gates [30],[29] but this is specific to Fermionic Hamiltonians, and is complementary to our work due to the mapping of the Hamiltonian into the spin representation being assumed as input to Kernpiler.

Unitary decomposition has been investigated mostly in a generic manner and separately from Hamiltonian mapping. Initial advancements, such as the quantum Shannon decomposition [41], demonstrated how arbitrary unitaries can be decomposed into single- and two-qubit unitaries. Recent studies have precisely quantified the number of gates required for unitary operations, notably demonstrating that any 3-qubit unitary can be decomposed into a maximum of 19 CNOT gates [26]. Although still above the theoretical minimum, these advances represent considerable progress. Additionally, numerical methods, while traditionally offering lower accuracy, provide intuitive trade-offs by significantly reducing gate counts, making them valuable for practical quantum computation applications [38], [35], [42], [49]. Overall, none of these general unitary decomposition methods take into account high level Hamiltonian information and therefore cannot adapt to high level structures of the unitary.

8 Conclusion

We introduced a novel compilation paradigm—leveraging partial Trotterization and strategic clustering of non-

commuting Hamiltonian terms—that substantially improves the computational efficiency and accuracy of quantum Hamiltonian simulation. Reinforcement learning (via MCTS) proved effective in discovering optimized gate structures, and the RL search frequently identified recurring CNOT scaffolds and entanglement motifs; these learned circuit patterns suggest heuristic or graph-based synthesis algorithms that do not rely on RL, and motivate studying how CNOT scaffolds relate to accuracy convergence of overparameterized circuits.

Acknowledgements

GL and ED were supported in part by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research through the Accelerated Research in Quantum Computing Program MACH-Q project., NSF CAREER Award No. CCF-2338773 and ExpandQISE Award No. OSI-2427020. GL is also supported by the Intel Rising Star Award. EM and EC were supported by the FY24 C2QA Postdoc Seed Funding Award from the Co-design Center for Quantum Advantage. EC was also supported in part by ARO MURI (award No. SCON-00005095), and DoE (BNL contract No. 433702). EG was supported by the NASA Academic Mission Services, Contract No. NNA16BD14C and the Intelligent Systems Research and Development-3 (ISRDS-3) Contract 80ARC020D0010 under Co-design Center for Quantum Advantage (C^2QA) under Contract No. DE-SC0012704. AS acknowledges support from the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Quantum Systems Accelerator.

References

[1] P. G. Anastasiou, Y. Chen, N. J. Mayhall, E. Barnes, and S. E. Economou, "Tetris-adapt-vqe: An adaptive algorithm that yields shallower, denser circuit ansätze," 2022.

[2] R. Babbush, N. Wiebe, J. McClean, J. McClain, H. Neven, and G. K.-L. Chan, "Low-Depth Quantum Simulation of Materials," *Physical Review X*, vol. 8, no. 1, p. 011044, Mar. 2018. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevX.8.011044

[3] C. W. Bauer, Z. Davoudi, N. Klco, and M. J. Savage, "Quantum simulation of fundamental particles and forces," *Nature Rev. Phys.*, vol. 5, no. 7, pp. 420–432, 2023.

[4] E. Campbell, "Random compiler for fast hamiltonian simulation," *Physical Review Letters*, vol. 123, no. 7, Aug. 2019. [Online]. Available: http://dx.doi.org/10.1103/PhysRevLett.123.070503

[5] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, I. D. Kivlichan, T. Menke, B. Peropadre, N. P. D. Sawaya, S. Sim, L. Veis, and A. Aspuru-Guzik, "Quantum chemistry in the age of quantum computing," *Chemical Reviews*, vol. 119, no. 19, pp. 10 856–10 915, aug 2019. [Online]. Available: https://doi.org/10.1021%2Facs.chemrev.8b00803

[6] A. M. Childs, A. Ostrander, and Y. Su, "Faster quantum simulation by randomization," *Quantum*, vol. 3, p. 182, Sep. 2019. [Online]. Available: http://dx.doi.org/10.22331/q-2019-09-02-182

[7] A. M. Childs, Y. Su, M. C. Tran, N. Wiebe, and S. Zhu, "Theory of trotter error with commutator scaling," *Physical Review X*, vol. 11, no. 1, Feb. 2021. [Online]. Available: http://dx.doi.org/10.1103/PhysRevX.11.011020

[8] A. M. Childs and N. Wiebe, "Hamiltonian simulation using linear combinations of unitary operations," *Quantum Information and Computation*, vol. 12, no. 11 & 12, pp. 901–924, Nov. 2012, publisher: Rinton Press. [Online]. Available: http://dx.doi.org/10.26421/QIC12.11-12-1

[9] A. Cowtan, S. Dilkes, R. Duncan, W. Simmons, and S. Sivarajah, "Phase gadget synthesis for shallow circuits," *Electronic Proceedings in Theoretical Computer Science*, vol. 318, p. 213–228, May 2020. [Online]. Available: http://dx.doi.org/10.4204/EPTCS.318.13

[10] A. Cowtan, W. Simmons, and R. Duncan, "A generic compilation strategy for the unitary coupled cluster ansatz," 2020. [Online]. Available: https://arxiv.org/abs/2007.10515

[11] E. Crane, K. C. Smith, T. Tomesh, A. Eickbusch, J. M. Martyn, S. Kühn, L. Funcke, M. A. DeMarco, I. L. Chuang, N. Wiebe, A. Schuckert, and S. M. Girvin, "Hybrid oscillator-qubit quantum processors: Simulating fermions, bosons, and gauge fields," 2024. [Online]. Available: https://arxiv.org/abs/2409.03747

[12] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," *International Journal of Theoretical Physics*, vol. 59, no. 12, p. 3804–3820, Nov. 2020. [Online]. Available: http://dx.doi.org/10.1007/s10773-020-04633-8

[13] C. M. Dawson and M. A. Nielsen, "The solovay-kitaev algorithm," 2005. [Online]. Available: https://arxiv.org/abs/quant-ph/0505030

[14] T. G. de Brugiere and S. Martiel, "Faster and shorter synthesis of hamiltonian simulation circuits," 2024. [Online]. Available: https://arxiv.org/abs/2404.03280

[15] C. Derby, J. Klassen, J. Bausch, and T. Cubitt, "Compact fermion-to-qubit mappings," *Physical Review B*, vol. 104, no. 3, p. 035118, 2021.

[16] C. Developers, "Cirq," May 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11398048

[17] S. Endo, S. C. Benjamin, and Y. Li, "Practical quantum error mitigation for near-future applications," *Physical Review X*, vol. 8, no. 3, Jul 2018. [Online]. Available: http://dx.doi.org/10.1103/PhysRevX.8.031027

[18] M. R. Geller and Z. Zhou, "Efficient error models for fault-tolerant architectures and the Pauli twirling approximation," *Physical Review A*, vol. 88, no. 1, p. 012314, Jul. 2013. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.88.012314

[19] K. Gui, T. Tomesh, P. Gokhale, Y. Shi, F. T. Chong, M. Martonosi, and M. Suchara, "Term grouping and travelling salesperson for digital quantum simulation," 2021. [Online]. Available: https://arxiv.org/abs/2001.05983

[20] N. Hatano and M. Suzuki, *Finding Exponential Product Formulas of Higher Orders*. Springer Berlin Heidelberg, Nov. 2005, p. 37–68. [Online]. Available: http://dx.doi.org/10.1007/11526216_2

[21] K. Hémery, K. Ghanem, E. Crane, S. L. Campbell, J. M. Dreiling, C. Figgatt, C. Foltz, J. P. Gaebler, J. Johansen, M. Mills, S. A. Moses, J. M. Pino, A. Ransford, M. Rowe, P. Siegfried, R. P. Stutz, H. Dreyer, A. Schuckert, and R. Nigmatullin, "Measuring the Loschmidt Amplitude for Finite-Energy Properties of the Fermi-Hubbard Model on an Ion-Trap Quantum Computer," *PRX Quantum*, vol. 5, no. 3, p. 030323, Aug. 2024, publisher: American Physical Society. [Online]. Available: https://link.aps.org/doi/10.1103/PRXQuantum.5.030323

[22] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," 2024.

[23] T. Kalajdzievski, C. Weedbrook, and P. Rebentrost, "Continuous-variable gate decomposition for the bose-hubbard model," *Physical Review A*, vol. 97, no. 6, Jun. 2018. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.97.062311

[24] E. Kaur, H. Shapourian, J. Zhao, M. Kilzer, R. Kompella, and R. Nejabati, "Optimized quantum circuit partitioning across multiple quantum processors," 2025. [Online]. Available: https://arxiv.org/abs/2501.14947

[25] N. Killoran, J. Izaac, N. Quesada, V. Bergholm, M. Amy, and C. Weedbrook, "Strawberry fields: A software platform for photonic quantum computing," *Quantum*, vol. 3, p. 129, Mar. 2019. [Online]. Available: http://dx.doi.org/10.22331/q-2019-03-11-129

[26] A. M. Krol and Z. Al-Ars, "Beyond quantum shannon decomposition: Circuit construction for n-qubit gates based on block-zxz decomposition," *Phys. Rev. Appl.*, vol. 22, p. 034019, Sep 2024. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevApplied.22.034019

[27] G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "Paulihedral: A generalized block-wise compiler optimization framework for quantum simulation kernels," 2021. [Online]. Available: https://arxiv.org/abs/2109.03371

[28] J. Liu, A. Gonzales, B. Huang, Z. H. Saleem, and P. Hovland, "Quclear: Clifford extraction and absorption for quantum circuit optimization," 2025. [Online]. Available: https://arxiv.org/abs/2408.13316

[29] Y. Liu, S. Che, J. Zhou, Y. Shi, and G. Li, "Fermihedral: On the Optimal Compilation for Fermion-to-Qubit Encoding," 3 2024.

[30] Y. Liu, K. Yao, J. Hong, J. Froustey, E. Rrapaj, C. Iancull, G. Li, and Y. Shi, "Hatt: Hamiltonian adaptive ternary tree for optimizing fermion-to-qubit mapping," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Mar. 2025, p. 143–157. [Online]. Available: http://dx.doi.org/10.1109/HPCA61900.2025.00022

[31] S. Lloyd, "Universal quantum simulators," *Science*, vol. 273, no. 5278, pp. 1073–1078, 1996.

[32] G. H. Low and I. L. Chuang, "Hamiltonian Simulation by Qubitization," *Quantum*, vol. 3, p. 163, Jul. 2019.

[33] L. A. Martínez-Martínez, T.-C. Yen, and A. F. Izmaylov, "Assessment of various hamiltonian partitionings for the electronic structure problem on a quantum computer using the trotter approximation," *Quantum*, vol. 7, p. 1086, Aug. 2023. [Online]. Available: http://dx.doi.org/10.22331/q-2023-08-16-1086

[34] J. R. McClean, K. J. Sung, I. D. Kivlichan, Y. Cao, C. Dai, E. S. Fried, C. Gidney, B. Gimby, P. Gokhale, T. Häner, T. Hardikar, V. Havlíček, O. Higgott, C. Huang, J. Izaac, Z. Jiang, X. Liu, S. McArdle, M. Neeley, T. O'Brien, B. O'Gorman, I. Ozfidan, M. D. Radin, J. Romero, N. Rubin, N. P. D. Sawaya, K. Setia, S. Sim, D. S. Steiger, M. Steudtner, Q. Sun, W. Sun, D. Wang, F. Zhang, and R. Babbush, "Openfermion: The electronic structure package for quantum computers," 2019. [Online]. Available: https://arxiv.org/abs/1710.07629

[35] Y. Nam, N. J. Ross, P.-H. Su, E. Younis, C. C. Iancu, W. Lavrijsen, and K. R. Brown, "Bqskit: The berkeley quantum synthesis toolkit," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2020, pp. 402–408.

[36] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.

[37] J. Paykin, A. T. Schmitz, M. Ibrahim, X.-C. Wu, and A. Y. Matsuura, "Pcoast: A pauli-based quantum circuit optimization framework," 2023. [Online]. Available: https://arxiv.org/abs/2305.10966

[38] P. Rakyta and Z. Zimborás, "Approaching the theoretical limit in quantum gate decomposition," *Quantum*, vol. 6, p. 710, May 2022. [Online]. Available: http://dx.doi.org/10.22331/q-2022-05-11-710

[39] A. T. Schmitz, N. P. D. Sawaya, S. Johri, and A. Y. Matsuura, "Graph optimization perspective for low-depth trotter-suzuki decomposition," 2023. [Online]. Available: https://arxiv.org/abs/2103.08602

[40] J. T. Seeley, M. J. Richard, and P. J. Love, "The bravyi–kitaev transformation for quantum computation of electronic structure," *The Journal of Chemical Physics*, vol. 137, no. 22, p. 224109, 2012.

[41] V. Shende, S. Bullock, and I. Markov, "Synthesis of quantum-logic circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 6, p. 1000–1010, Jun. 2006. [Online]. Available: http://dx.doi.org/10.1109/TCAD.2005.855930

[42] E. Smith, M. G. Davis, J. Larson, E. Younis, L. B. Oftelie, W. Lavrijsen, and C. Iancu, "Leap: Scaling numerical optimization based synthesis using an incremental approach," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, p. 1–23, Feb. 2023. [Online]. Available: http://dx.doi.org/10.1145/3548693

[43] T. J. Stavenger, E. Crane, K. C. Smith, C. T. Kang, S. M. Girvin, and N. Wiebe, "C2QA - Bosonic Qiskit," in *26th IEEE High Performance Extreme Computing*, 9 2022.

[44] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[45] E. van den Berg and K. Temme, "Circuit optimization of hamiltonian simulation by simultaneous diagonalization of pauli clusters," *Quantum*, vol. 4, p. 322, Sep. 2020. [Online]. Available: http://dx.doi.org/10.22331/q-2020-09-12-322

[46] J. J. Wallman and J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling," *Physical Review A*, vol. 94, no. 5, Nov 2016. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.94.052325

[47] J. J. Wallman and J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling," *Phys. Rev. A*, vol. 94, p. 052325, Nov 2016. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.94.052325

[48] A. Winick, J. J. Wallman, D. Dahlen, I. Hincks, E. Ospadov, and J. Emerson, "Concepts and conditions for error suppression through randomized compiling," 12 2022.

[49] E. Younis, K. Sen, K. Yelick, and C. Iancu, "Qfast: Quantum synthesis using a hierarchical continuous circuit space," University of California, Berkeley, EECS Dept., Tech. Rep. UCB/EECS-2020-53, 2020. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2020/EECS-2020-53.pdf